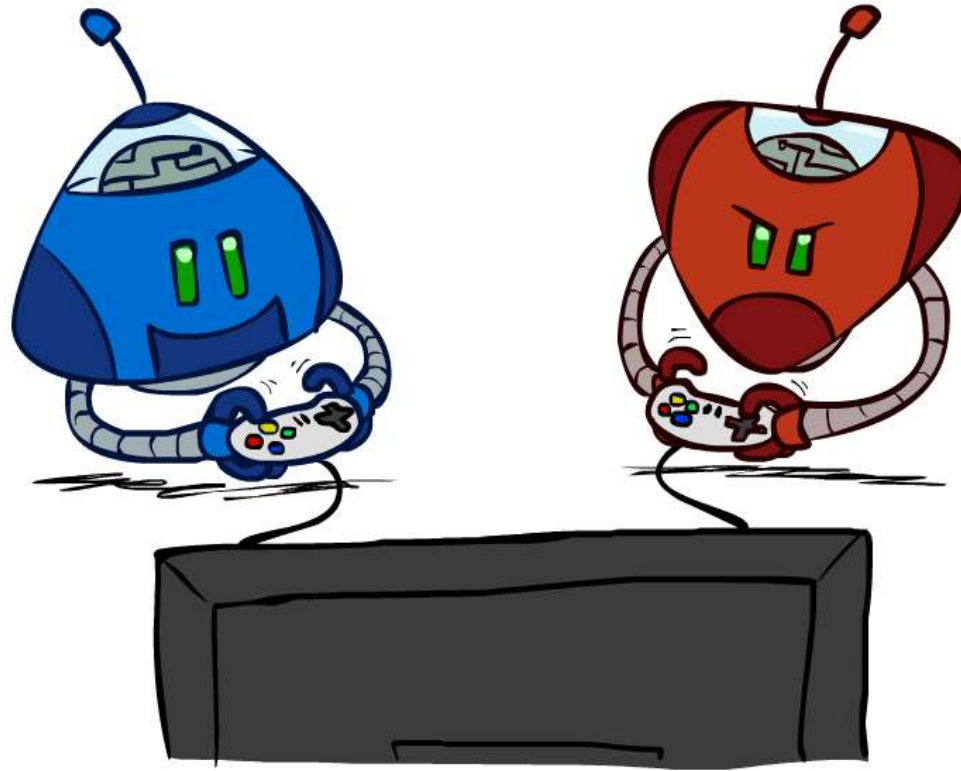


AIT2004: Cơ sở Trí tuệ nhân tạo

Tìm kiếm có đối thủ và cây tìm kiếm trò chơi



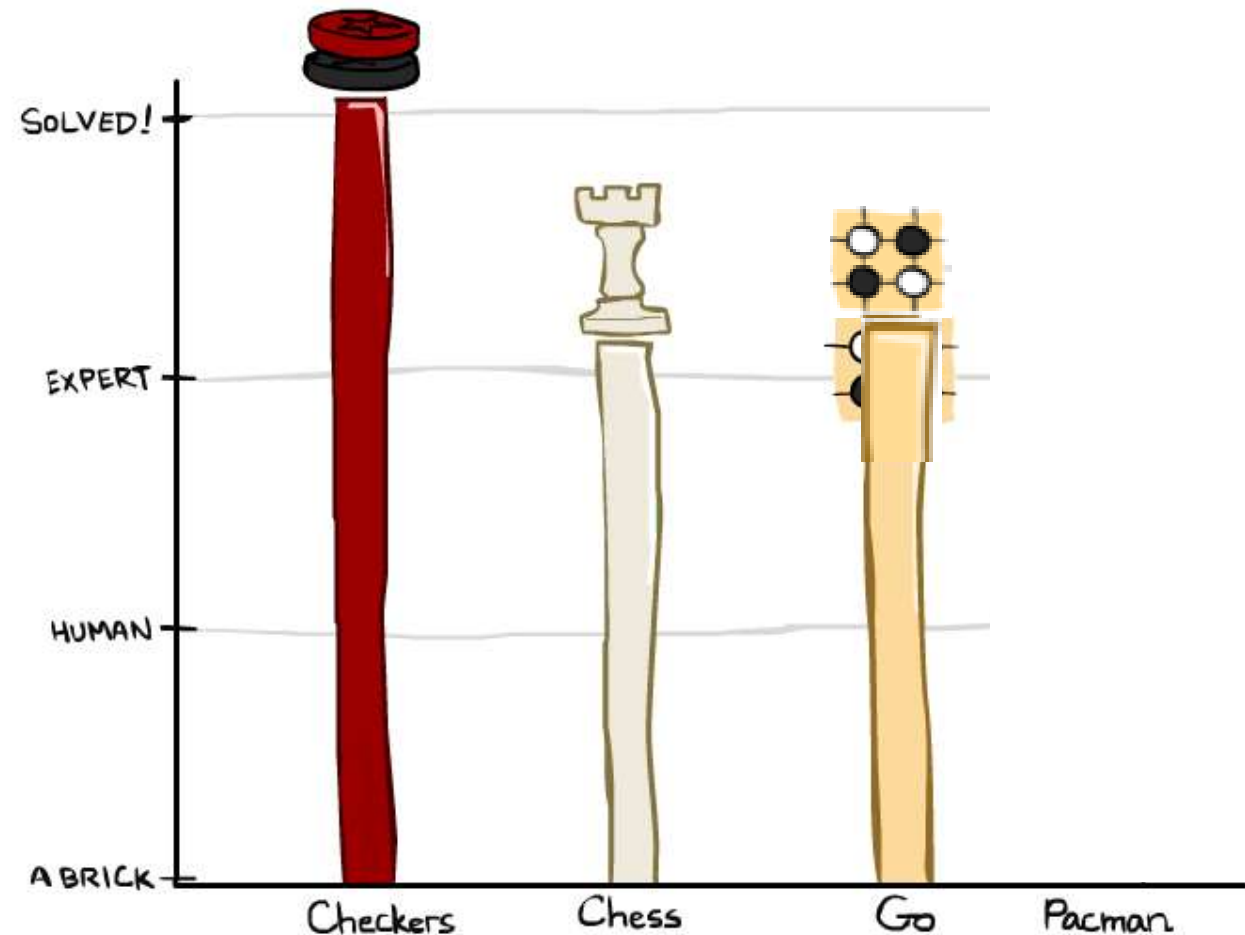
[Sử dụng một số slides của Dan Klein và Pieter Abbeel từ CS188 Intro to AI at UC Berkeley]

Học liệu

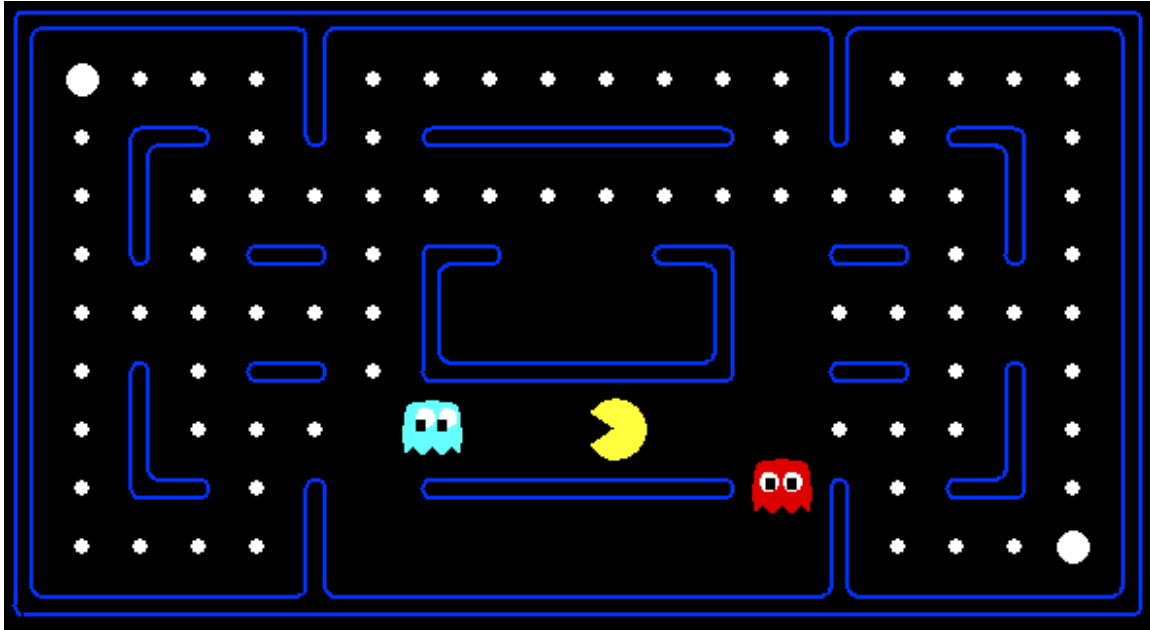
- Đinh Mạnh Tường, *Trí tuệ nhân tạo – Cách tiếp cận hiện đại*, NXB Khoa học kỹ thuật, 2024
 - Chương 7. Tìm kiếm có đối thủ
- Wolfgang Ertel, *Introduction to Artificial Intelligence (2nd ed)*, Springer 2017
 - Chapter 6. Search, Games and Problem Solving
- Russell & Norvig, *AI: A Modern Approach*, 4th ed., Pearson 2020
 - Chapter 5. Adversarial Search and Games

Game Playing State-of-the-Art

- **Cờ đam (Checkers):** 1950: Chương trình máy tính đầu tiên. 1994: Máy tính vô địch: Chinook kết thúc 40 năm thống trị của nhà vô địch thế giới Marion Tinsley sử dụng các thế cờ tàn gồm 8 quân. 2007: Checkers đã được giải quyết !
- **Cờ vua (Chess):** 1997: Deep Blue đánh bại nhà vô địch thế giới Gary Kasparov trong trận đấu 6 ván. Deep Blue tính 200 triệu nước đi trong một giây, sử dụng các ước lượng phức tạp và phương pháp mở rộng cây tìm kiếm (không công bố) lên đến 40 nước đi. Các chương trình ngày nay còn tốt hơn thế.
- **Cờ vây (Go):** 2016: Alpha GO đánh bại vô địch thế giới. Sử dụng tìm kiếm Monte Carlo và hàm ước lượng được huấn luyện.
- **Pacman**



Trò chơi



Hành vi được tính toán

Có đối thủ



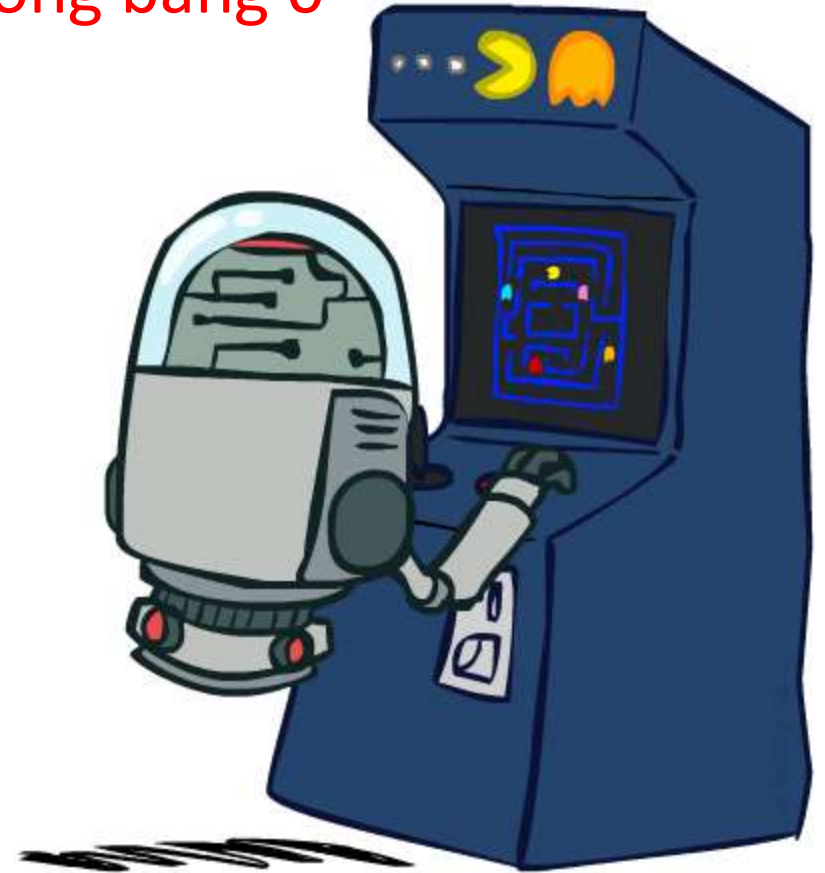
Các loại trò chơi

- Có rất nhiều kiểu trò chơi!
 - **Tất định** hay ngẫu nhiên?
 - Một, hai, ba hay nhiều người chơi?
 - Tổng bằng 0 (đối kháng)?
 - Có thông tin đầy đủ (biết toàn bộ trạng thái) không?
- Cần các thuật toán tính ra **chiến lược** gợi ý nước đi cho mỗi trạng thái của trò chơi

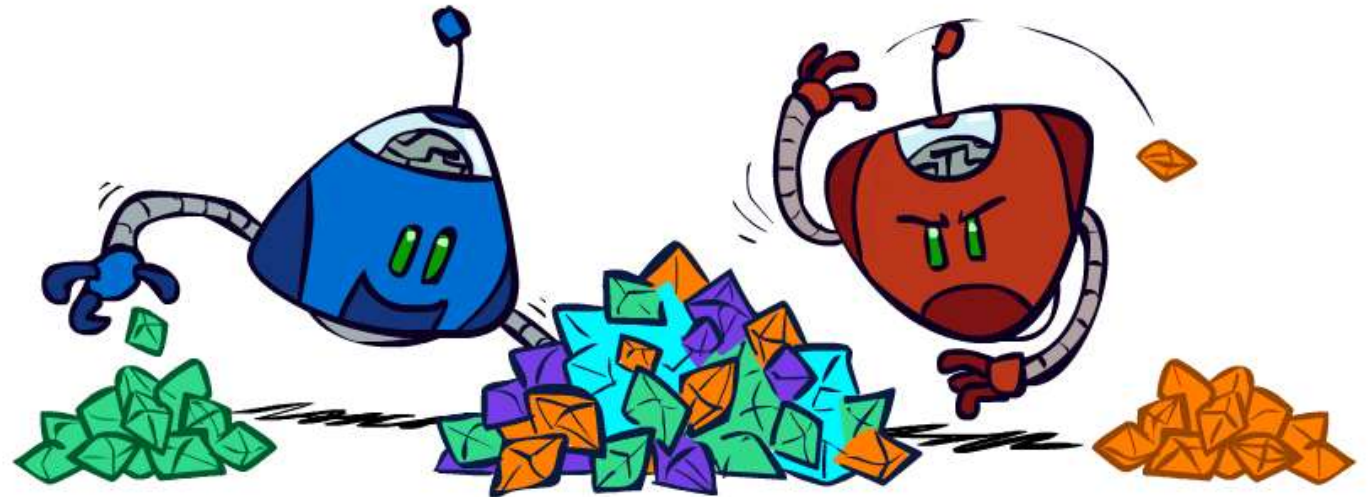


Trò chơi “tiêu chuẩn”

- Tất định, quan sát được, 2 người chơi, theo lượt, tổng bằng 0
- Một cách mô hình hóa:
 - Tập trạng thái: S (bắt đầu từ s_0)
 - Người chơi : $P=\{1, 2\}$
 - Tập hành động: A (phụ thuộc người chơi và trạng thái)
 - Hàm chuyển trạng thái: $S \times A \rightarrow S$
 - Hàm kiểm tra kết thúc : $S \rightarrow \{t, f\}$
 - Hàm đánh giá trạng thái kết thúc (lợi ích):
 $S \times P \rightarrow R$
- Giải pháp là một chiến lược $S \rightarrow A$
 - Mỗi người chơi dùng một hàm để tính hành động khi đến lượt



Trò chơi có tổng bằng không (Zero-sum games)



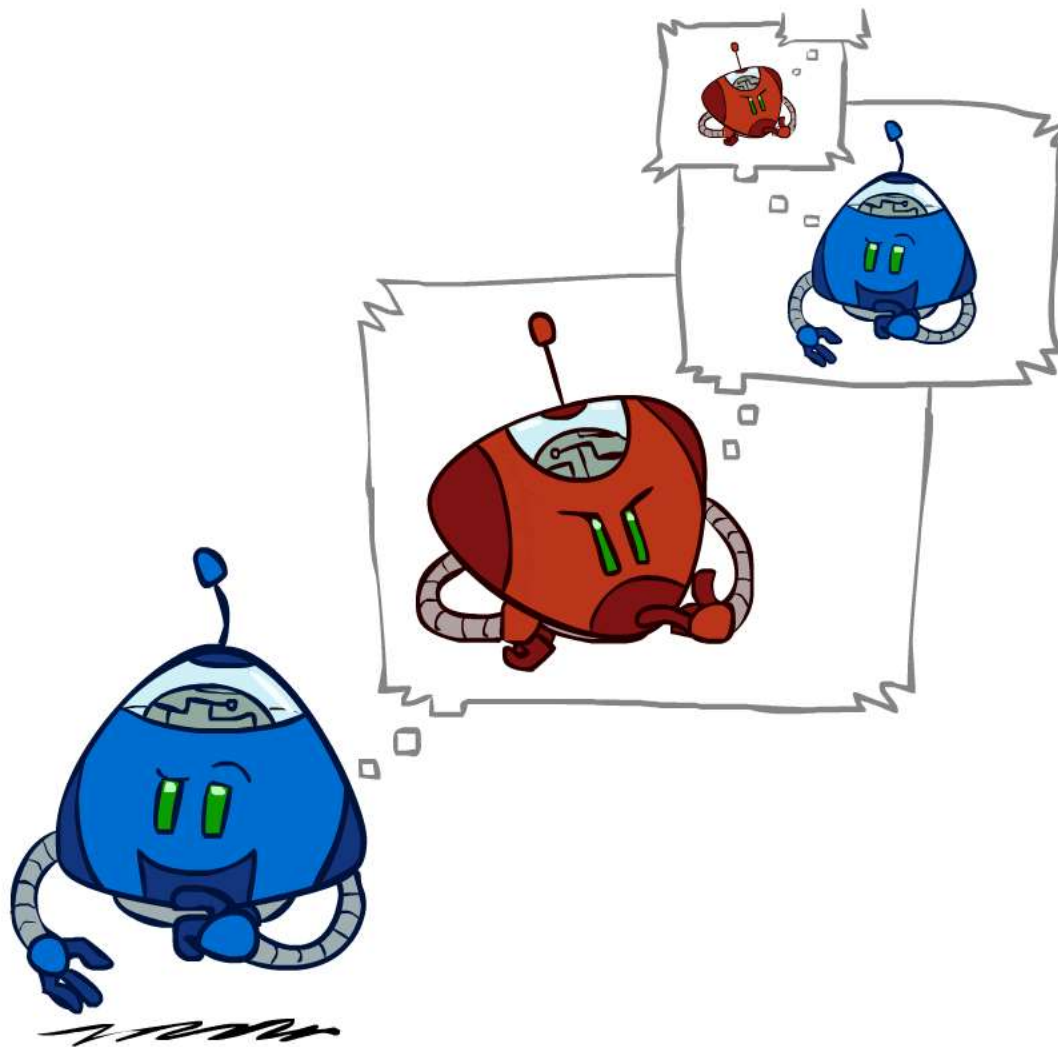
■ Trò chơi có tổng bằng không

- Các người chơi đánh giá tính lợi ích (giá trị) trái ngược nhau
- Giá trị lợi ích: Một người chơi cực đại hóa, người còn lại cực tiểu hóa
- Đối kháng = cạnh tranh trực tiếp

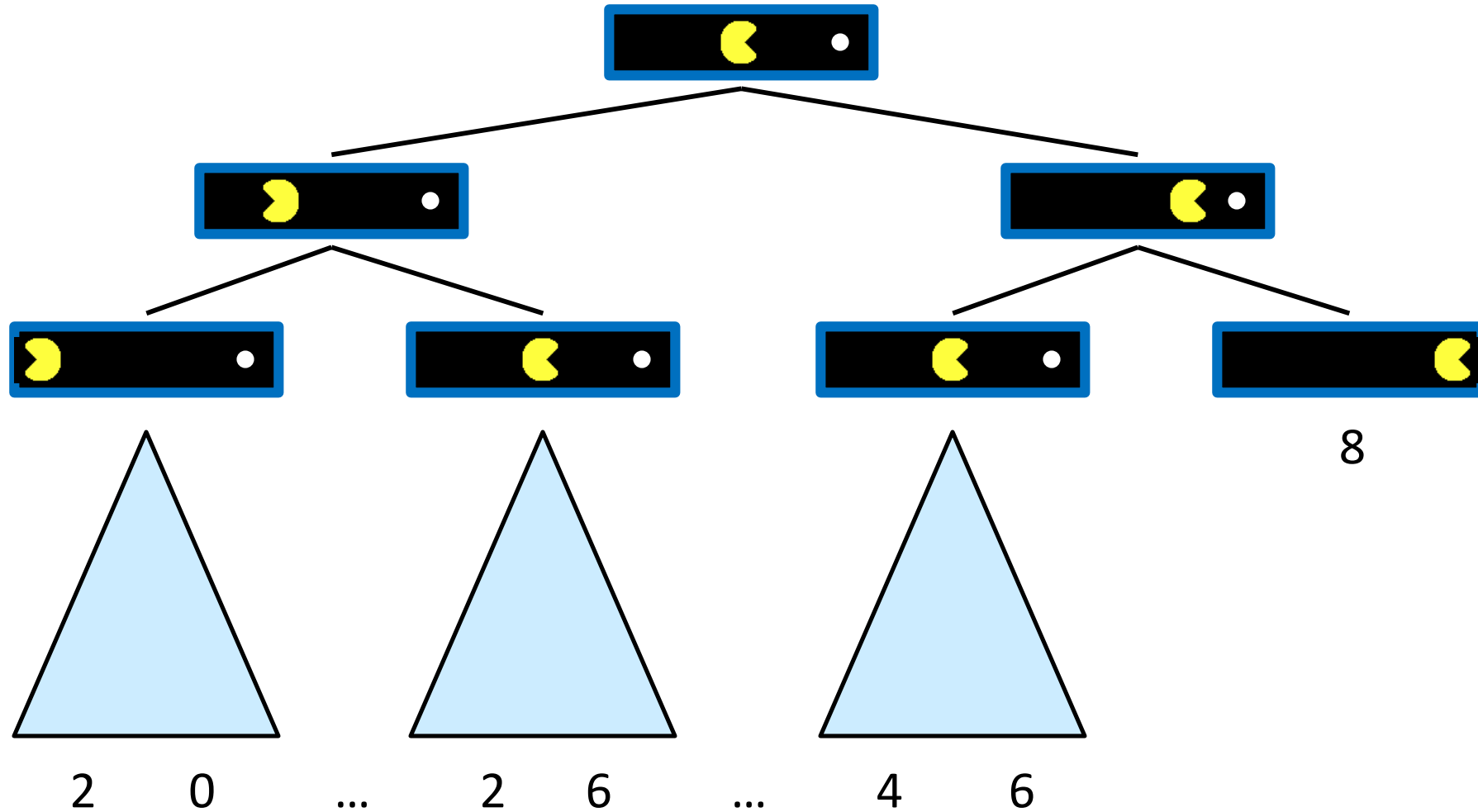
■ Trò chơi tổng quát

- Các người chơi đánh giá tính lợi ích độc lập với nhau
- Hợp tác, không quan tâm, cạnh tranh, và nhiều kiểu khác

Tìm kiếm có đối thủ

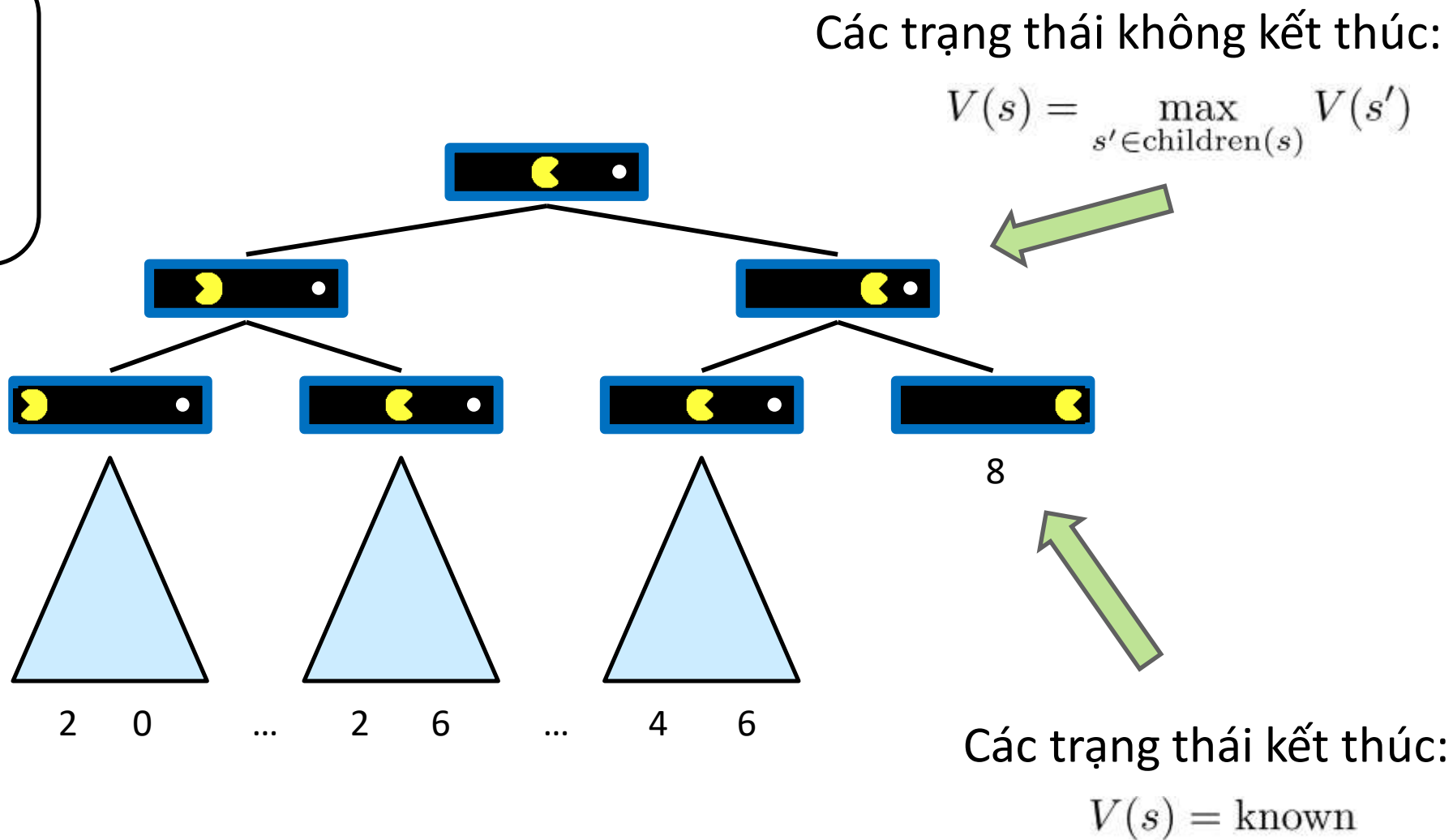


Cây tìm kiếm của một tác tử

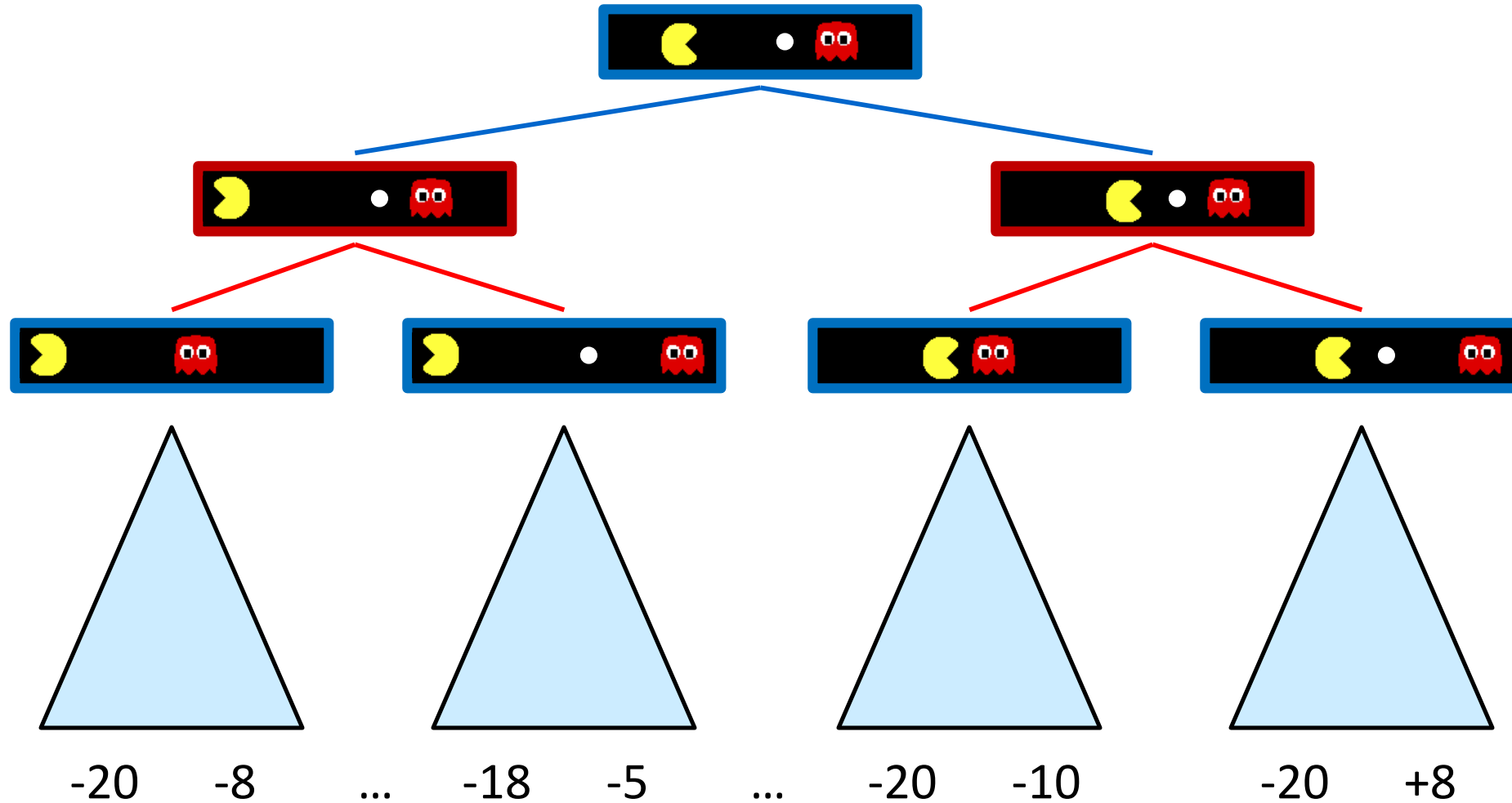


Giá trị của trạng thái

Giá trị của trạng thái: Giá trị tốt nhất có thể đạt được từ trạng thái đó



Cây tìm kiếm khi có đối thủ



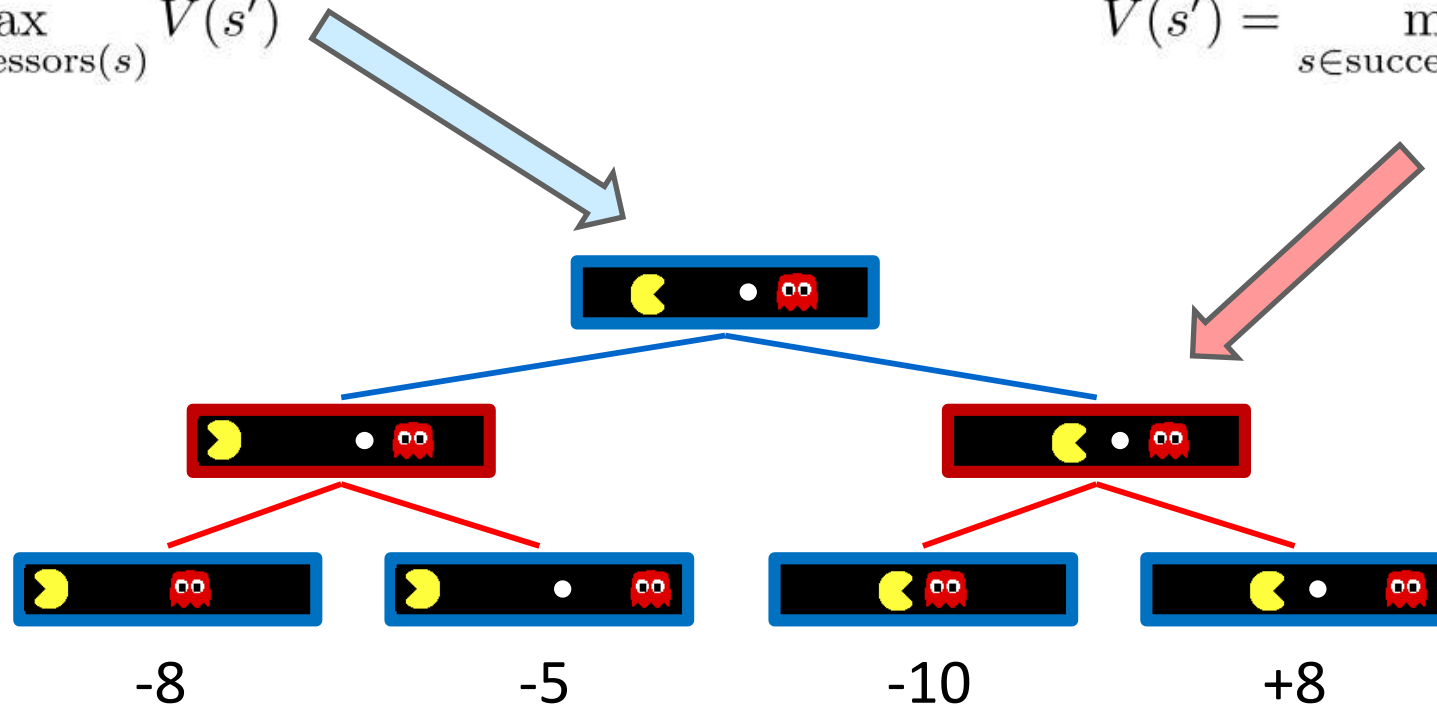
Giá trị Minimax

Trạng thái của tác tử:

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

Trạng thái của tác tử đối thủ:

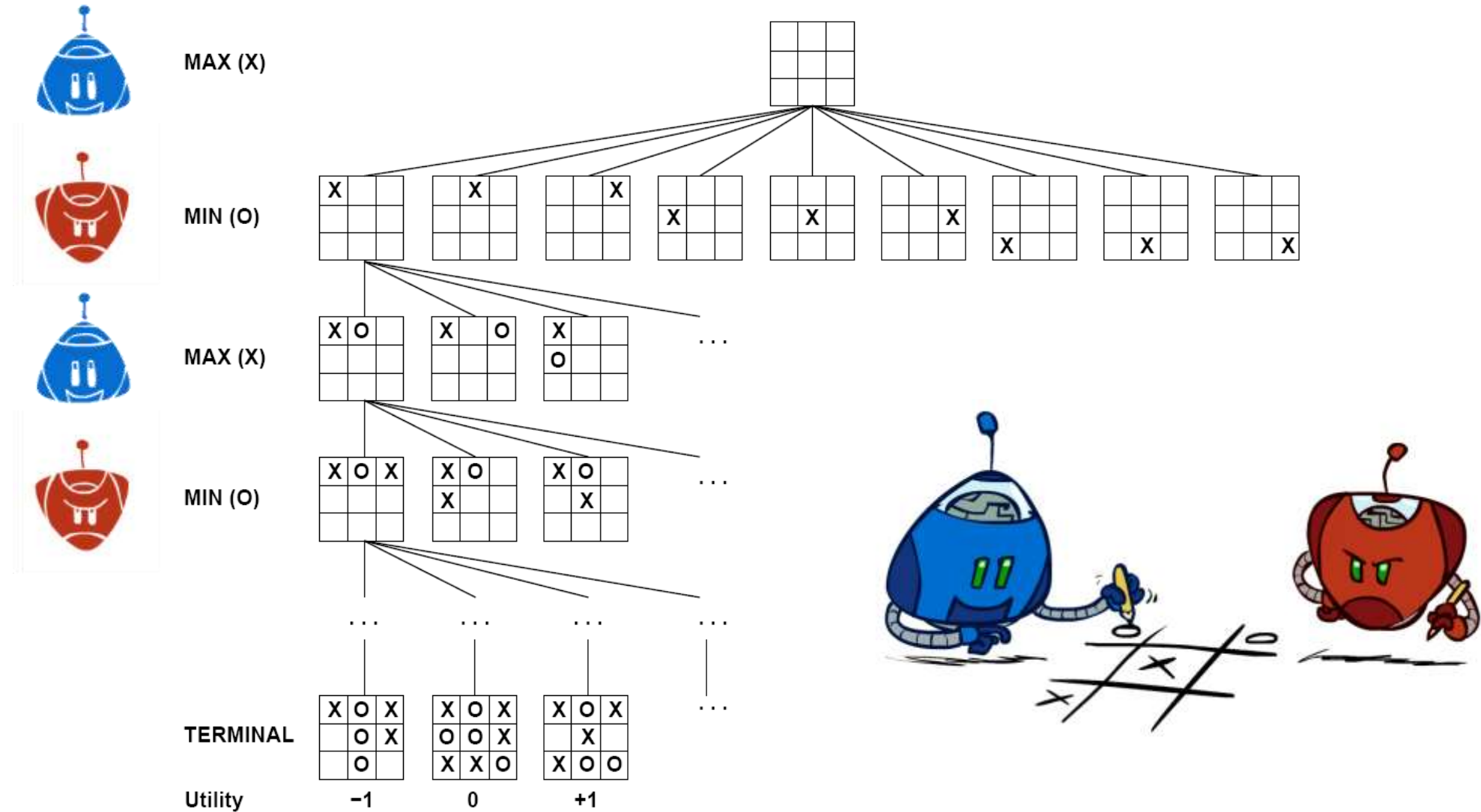
$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$



Trạng thái kết thúc:

$$V(s) = \text{known}$$

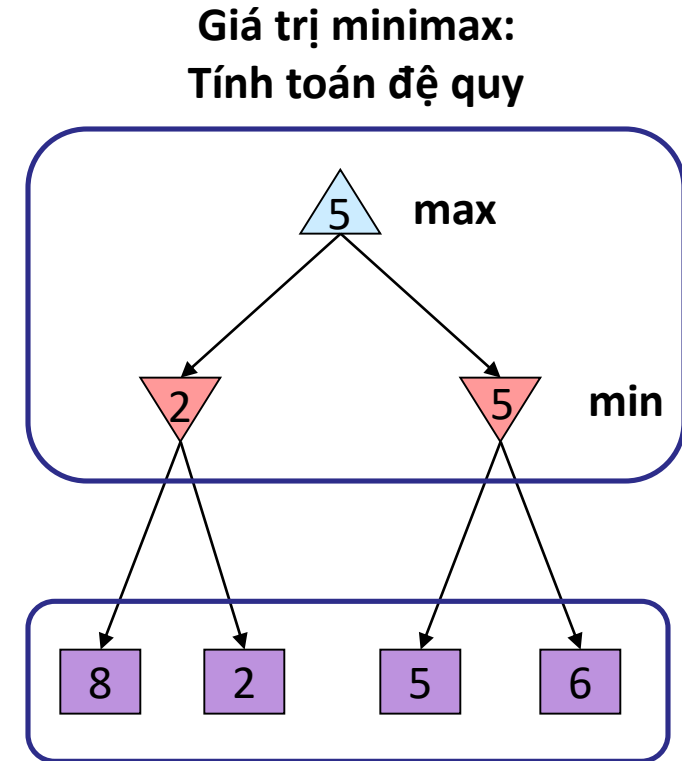
Cây tìm kiếm của Tic-Tac-Toe



Tìm kiếm có đối thủ (Minimax)

- Trò chơi tất định có tổng bằng không:
 - Tic-tac-toe, cờ vua, cờ đam
 - Một người chơi cực đại giá trị
 - Người còn lại cực tiểu giá trị
- Tìm kiếm minimax:
 - Cây tìm kiếm trạng thái
 - Người chơi lần lượt
 - **Tính giá trị minimax: giá trị tốt nhất đạt được khi đối thủ chơi hợp lý (tối ưu)**

“Tối thiểu hóa giá trị tối đa của đối thủ”



Các giá trị kết thúc:
được định nghĩa từ luật chơi

Cài đặt minimax

def max-value(state):

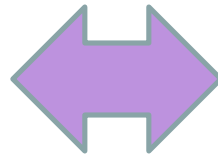
 initialize $v = -\infty$

 for each successor of state:

$v = \max(v, \text{min-value}(\text{successor}))$

 return v

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$



def min-value(state):

 initialize $v = +\infty$

 for each successor of state:

$v = \min(v, \text{max-value}(\text{successor}))$

 return v

$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$

Refactor

```
def value(state):
```

if the state is a terminal state: return the state's utility

if the next agent is MAX: return max-value(state)

if the next agent is MIN: return min-value(state)

```
def max-value(state):
```

initialize $v = -\infty$

for each successor of state:

$v = \max(v, \text{value}(\text{successor}))$

return v

```
def min-value(state):
```

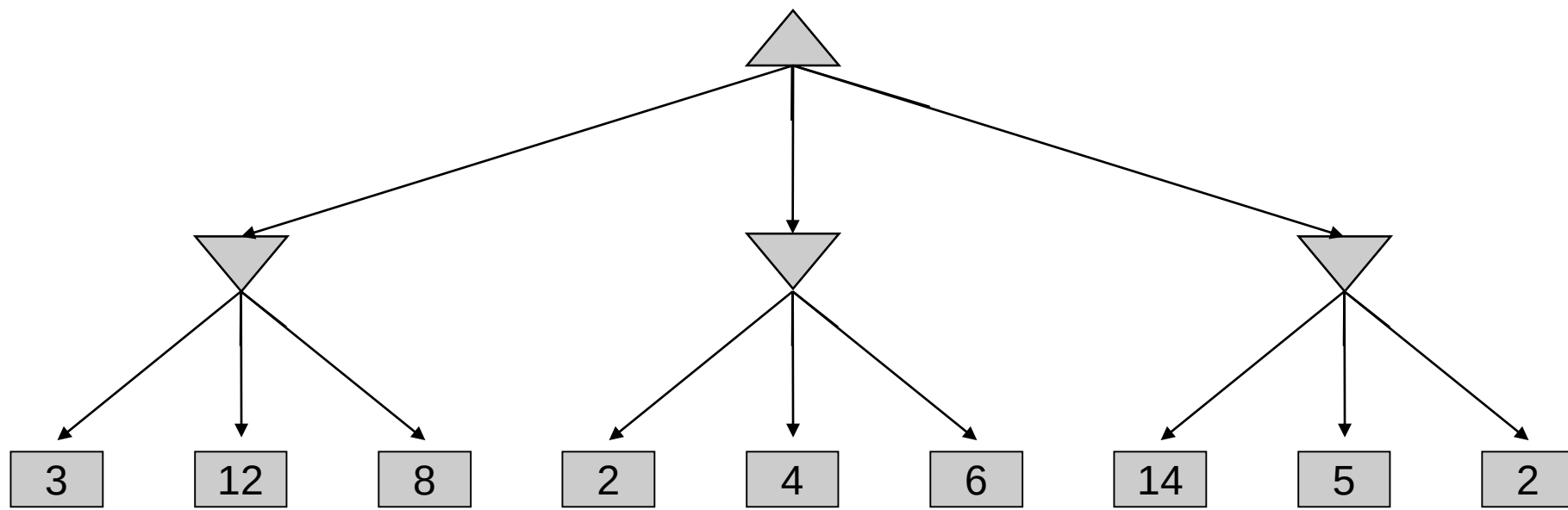
initialize $v = +\infty$

for each successor of state:

$v = \min(v, \text{value}(\text{successor}))$

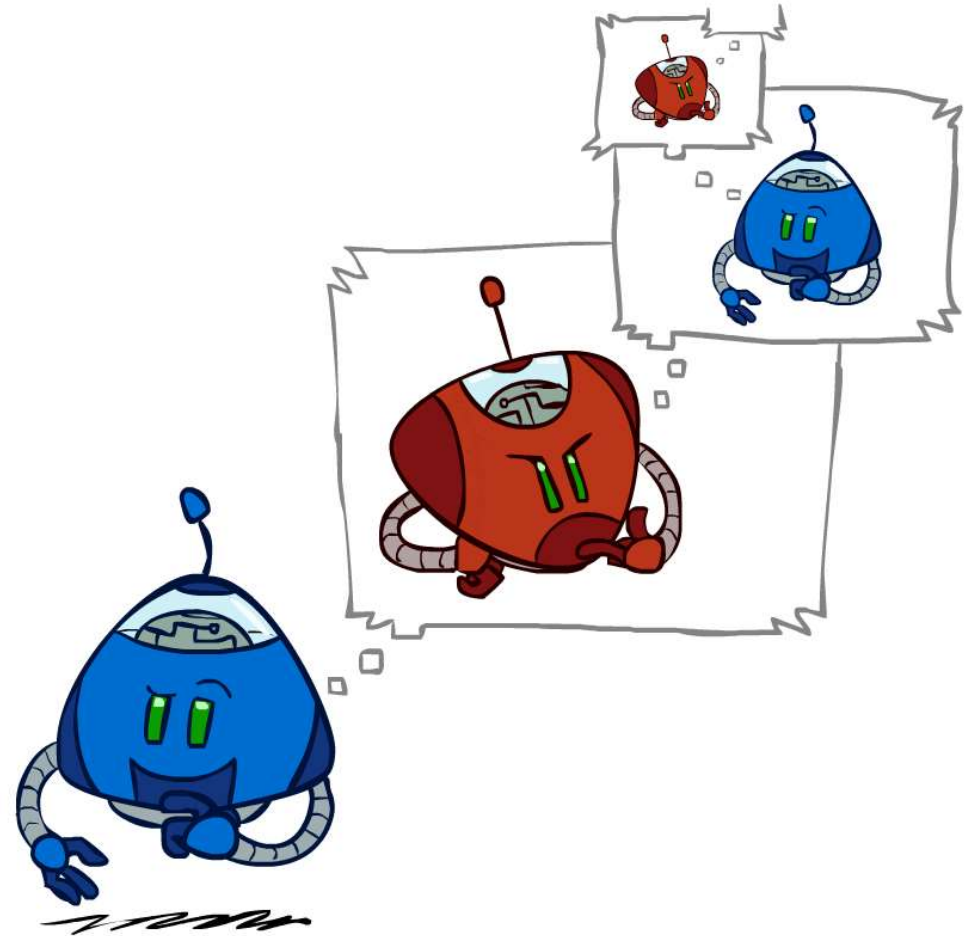
return v

Ví dụ: Minimax



Tính chất của Minimax

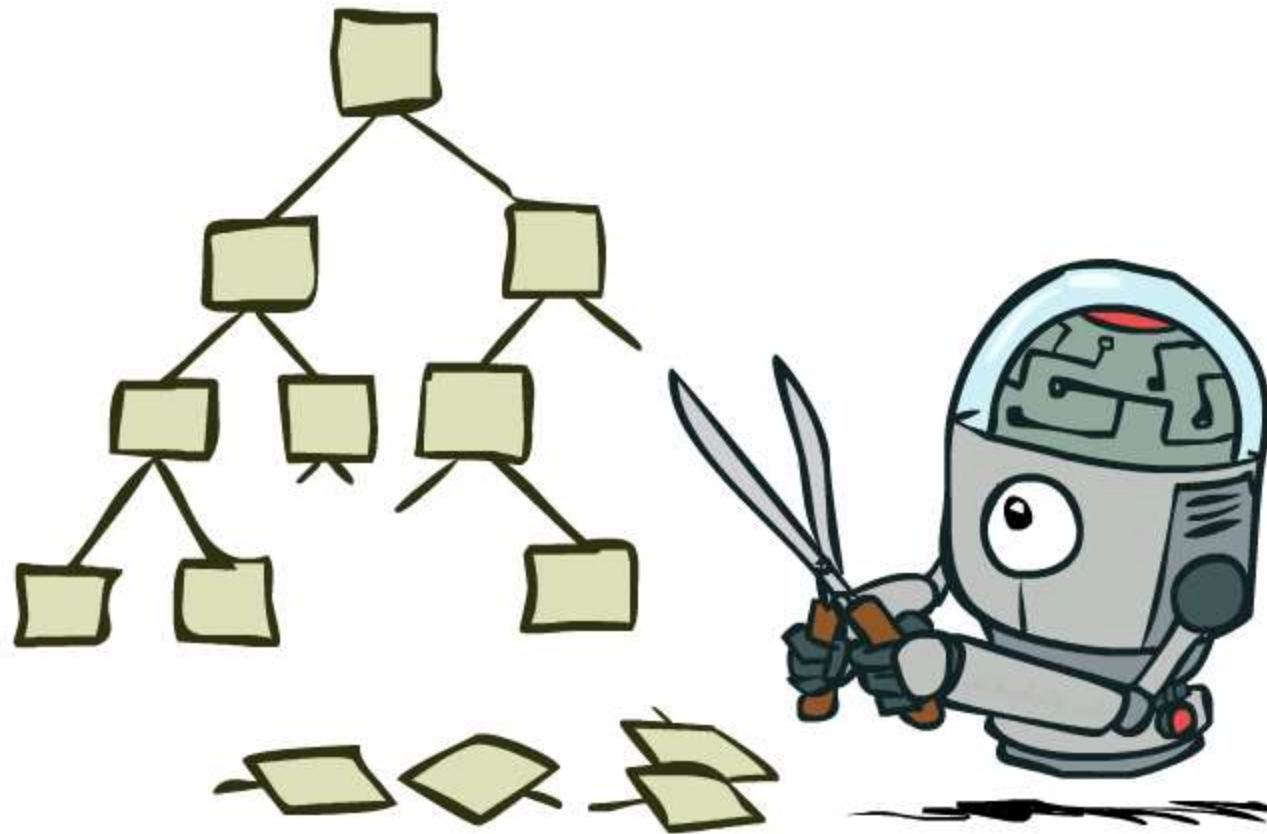
- Độ phức tạp
 - Giống DFS
 - Thời gian: $O(b^m)$
 - Không gian: $O(bm)$
- Ví dụ: Cờ vua, $b \approx 35$, $m \approx 100$
 - Tính giải pháp chính xác = không khả thi
 - Câu hỏi: Có cần mở rộng toàn bộ cây tìm kiếm ???



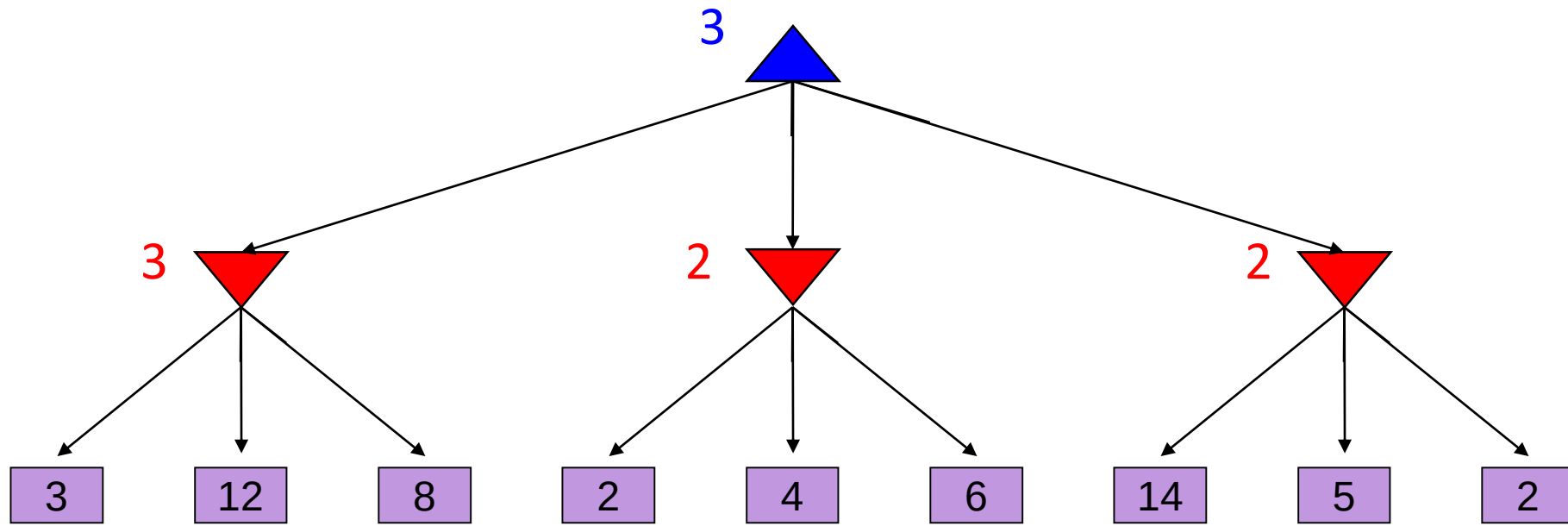
Giới hạn nguồn lực



Cắt nhánh cây tìm kiếm

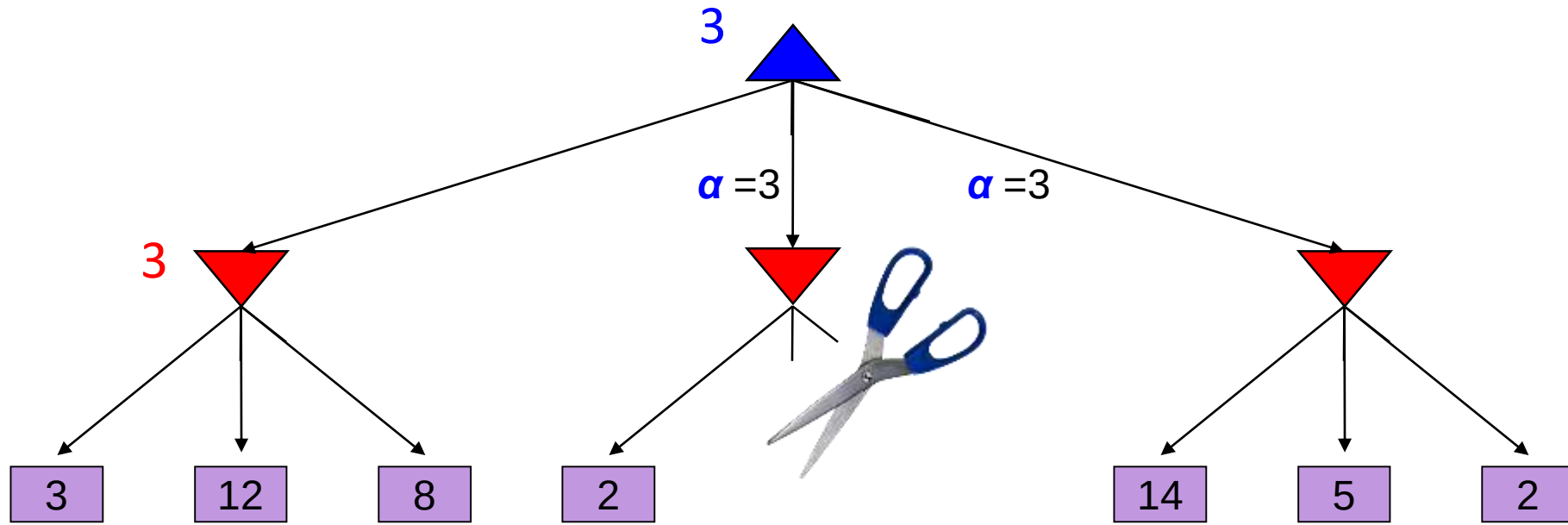


Ví dụ: Minimax



Ví dụ: cắt nhánh Alpha-Beta

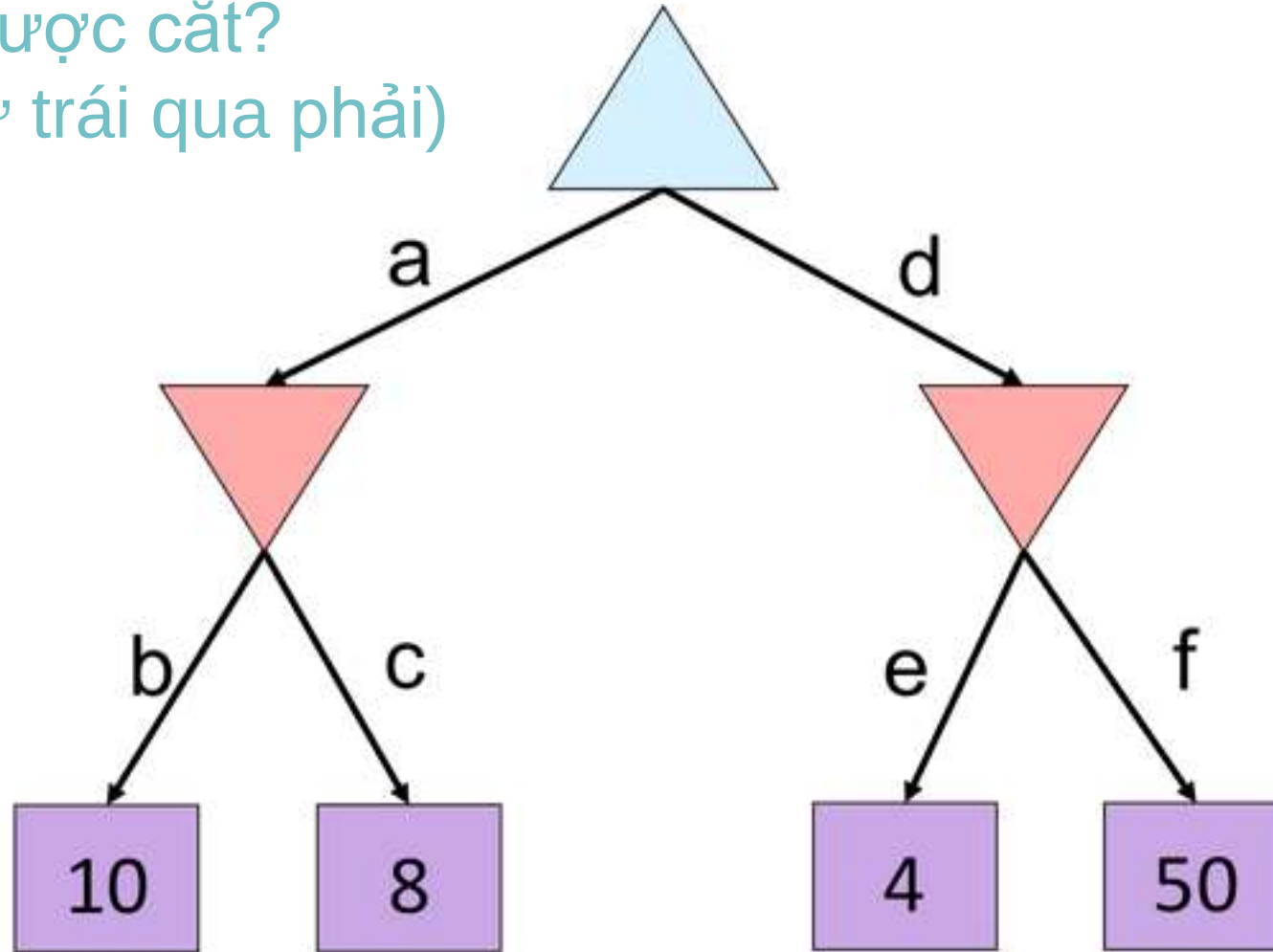
α = best option so far from any
MAX node on this path



The order of generation matters: more pruning
is possible if good moves come first

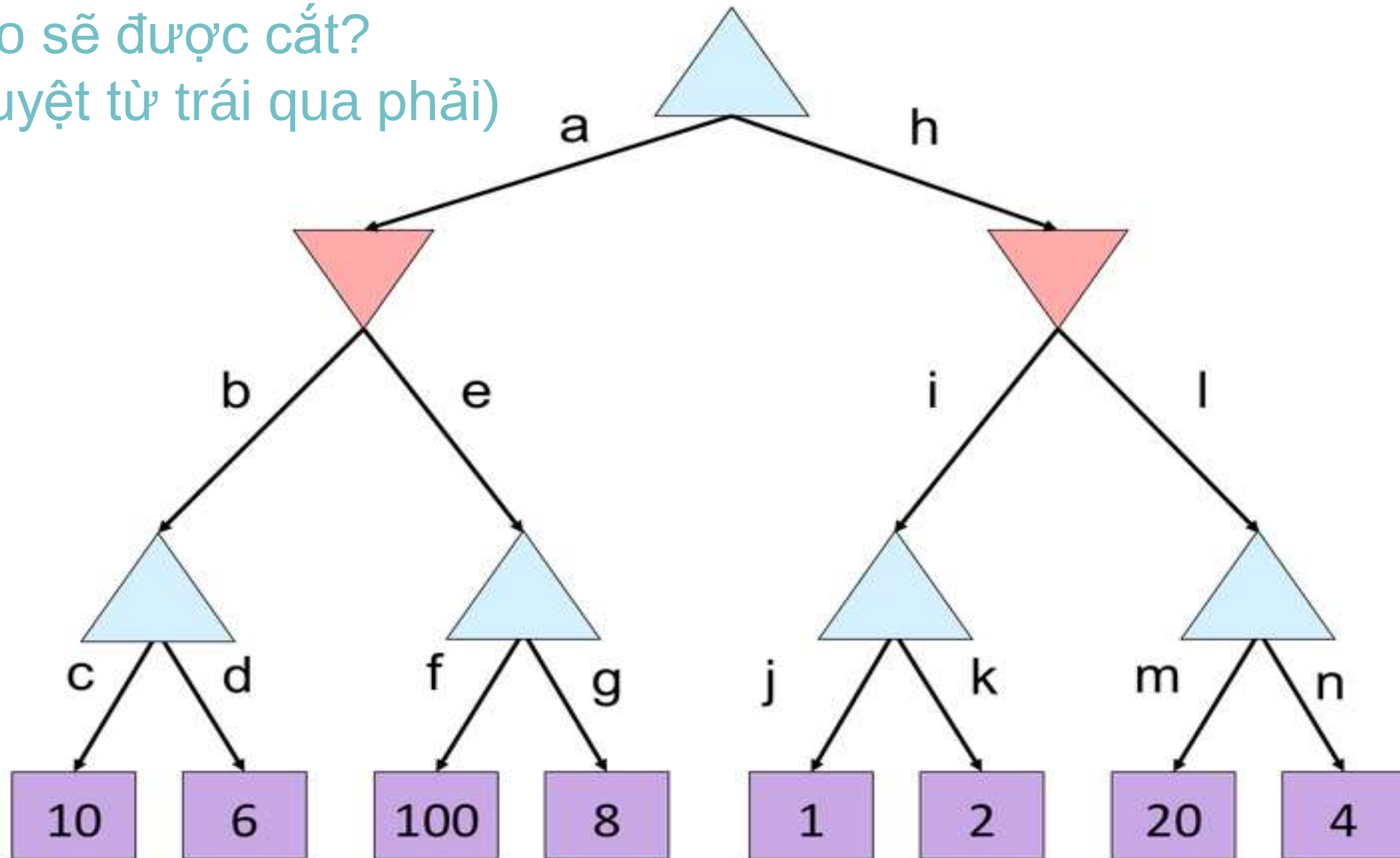
Ví dụ

Nhánh nào sẽ được cắt?
(Giả sử duyệt từ trái qua phải)

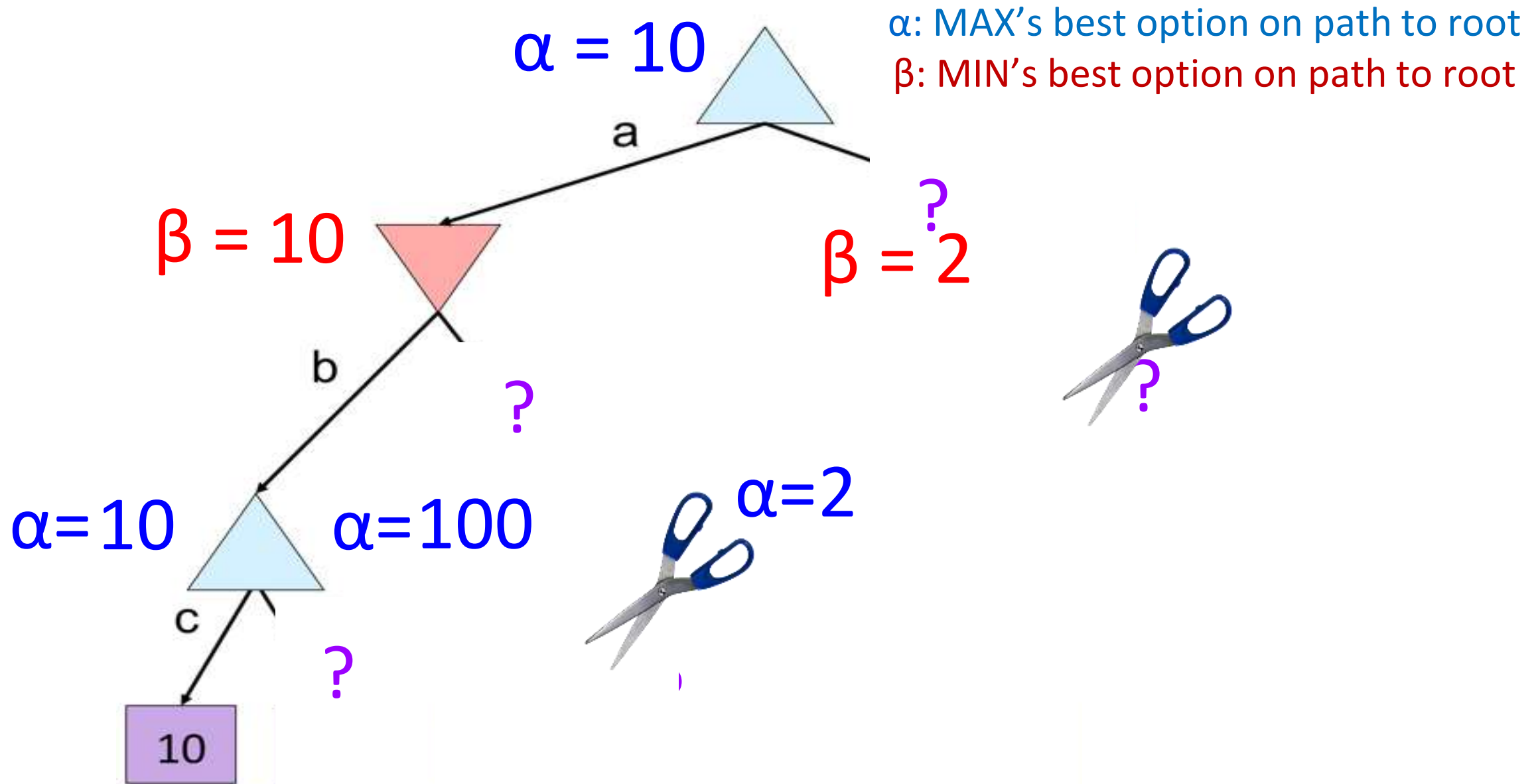


Ví dụ

Nhánh nào sẽ được cắt?
(Giả sử duyệt từ trái qua phải)



Cắt nhánh Alpha-Beta



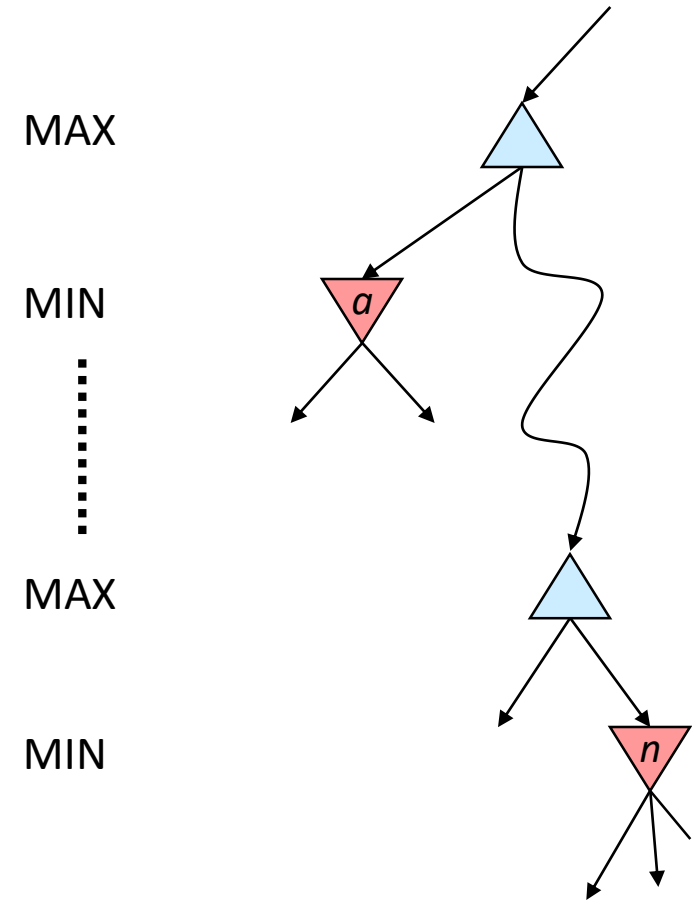
Thuật toán cắt nhánh Alpha-Beta

■ Xét trường hợp mở rộng nút MIN

- Ta đang tính giá trị MIN tại nút n
- Cần duyệt qua các nút con của n
- *Giá trị min giảm dần*
- Ai quan tâm đến giá trị của n ? MAX
- Gọi a là giá trị tốt nhất mà MAX có thể đạt được dọc theo đường đi hiện tại tính từ gốc của cây
- Nếu giá trị của n nhỏ hơn a , MAX sẽ không bao giờ đi vào n → có thể bỏ qua các con khác của n (vì các con khác không tăng được giá trị min)

■ Trường hợp mở rộng nút MAX cũng tương tự

- Giá trị max tăng dần, nếu vượt quá giá trị b là giá trị tốt nhất MIN đạt được dọc theo đường đi hiện tại tính từ gốc → bỏ qua các nút con khác



Cài đặt Alpha-Beta

α : MAX's best option on path to root
 β : MIN's best option on path to root

```
def max-value(state,  $\alpha$ ,  $\beta$ ):  
    initialize  $v = -\infty$   
    for each successor of state:  
         $v = \max(v, \text{min-value}(\text{successor}, \alpha, \beta))$   
        if  $v \geq \beta$   
            return  $v$   
         $\alpha = \max(\alpha, v)$   
    return  $v$ 
```

```
def min-value(state,  $\alpha$ ,  $\beta$ ):  
    initialize  $v = +\infty$   
    for each successor of state:  
         $v = \min(v, \text{max-value}(\text{successor}, \alpha, \beta))$   
        if  $v \leq \alpha$   
            return  $v$   
         $\beta = \min(\beta, v)$   
    return  $v$ 
```

Alpha-Beta

α : MAX's best option on path to root

β : MIN's best option on path to root

def max-value(state, α , β):

 initialize $v = -\infty$

 for each successor of state:

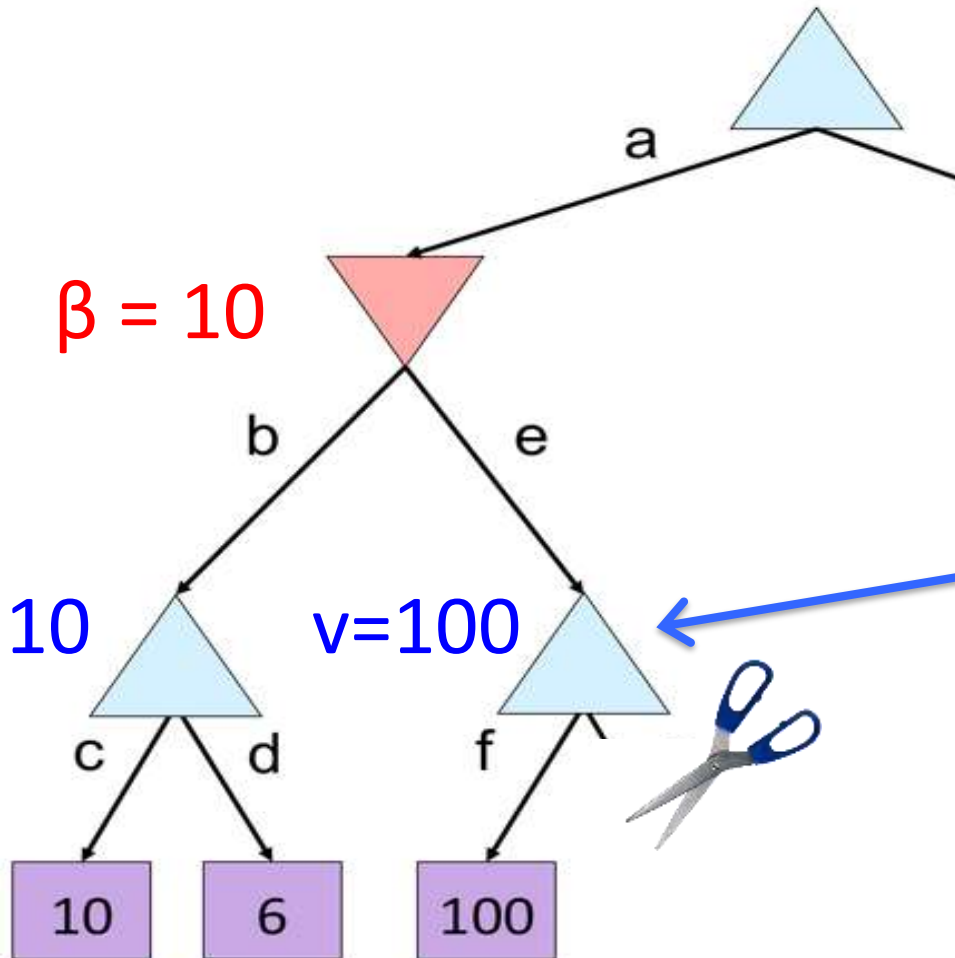
$v = \max(v, \text{min-value}(\text{successor}, \alpha, \beta))$

if $v \geq \beta$

return v

$\alpha = \max(\alpha, v)$

return v



Alpha-Beta

α : MAX's best option on path to root

β : MIN's best option on path to root

def min-value(state, α , β):

 initialize $v = +\infty$

 for each successor of state:

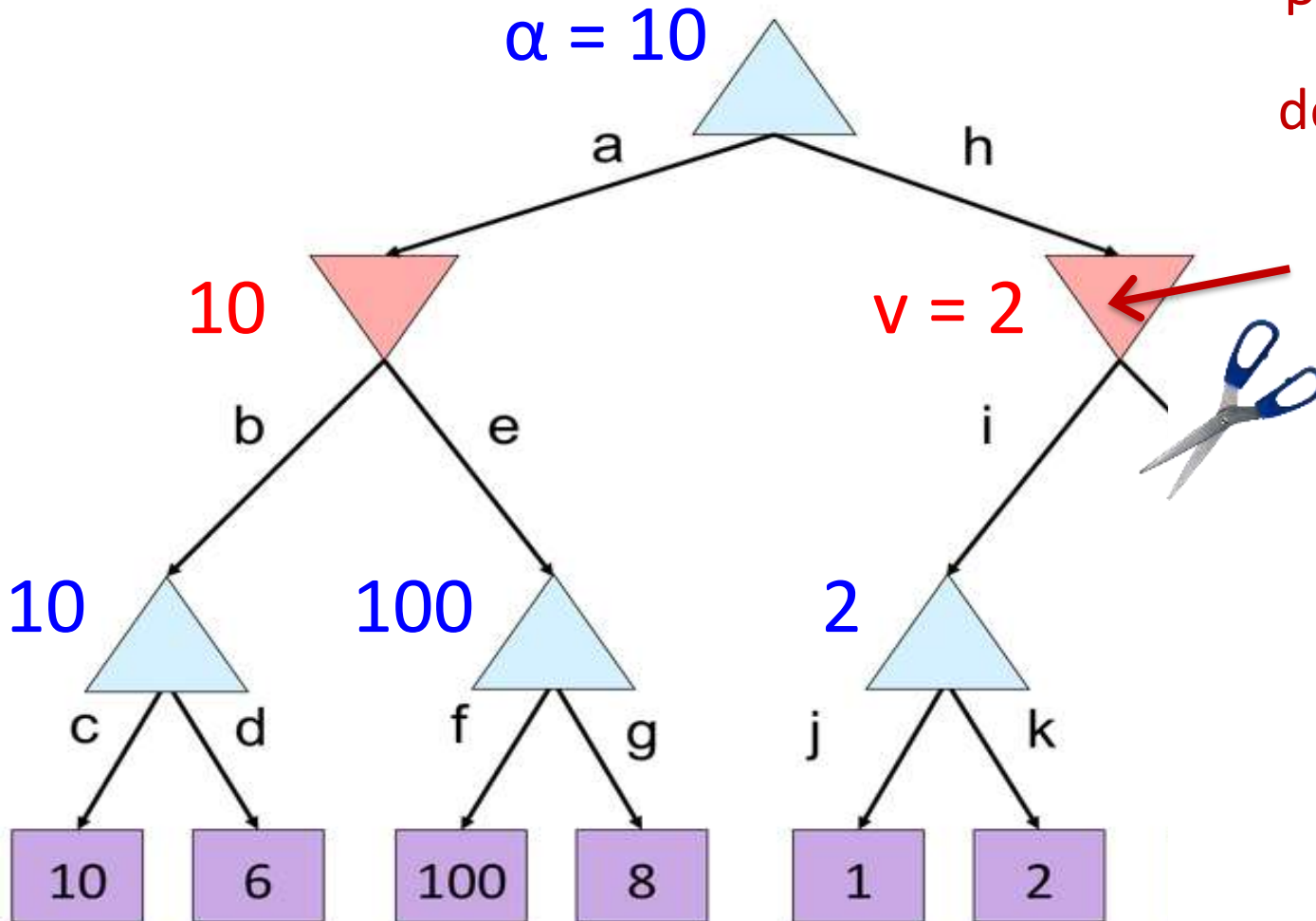
$v = \min(v, \text{max-value}(\text{successor}, \alpha, \beta))$

 if $v \leq \alpha$

 return v

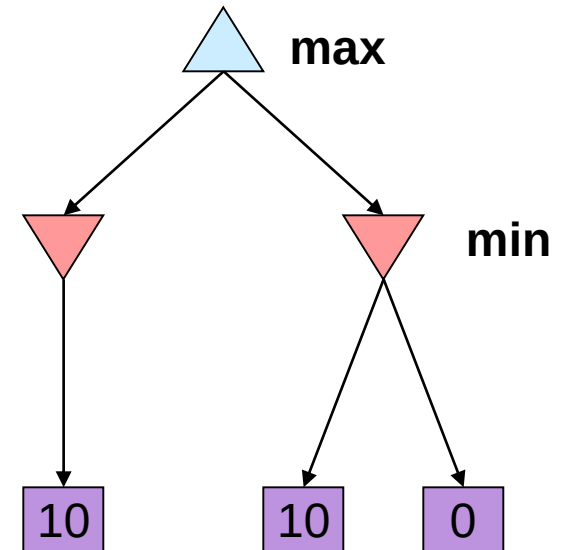
$\beta = \min(\beta, v)$

 return v



Tính chất của cắt nhánh Alpha-Beta

- **Không thay đổi** giá trị minimax của nút gốc !!!
- Giá trị của các nút ở giữa có thể sai
 - Lưu ý: giá trị các nút con của gốc có thể sai
- Thứ tự duyệt các nút con thay đổi khả năng mở rộng các nút khác
- Thứ tự tối ưu:
 - Thời gian giảm xuống còn $O(b^{m/2})$
 - Tăng gấp đôi độ sâu hiệu quả !
- Vẫn không đủ mạnh để đảm bảo tìm kiếm đầy đủ (ví dụ cho cờ vua)

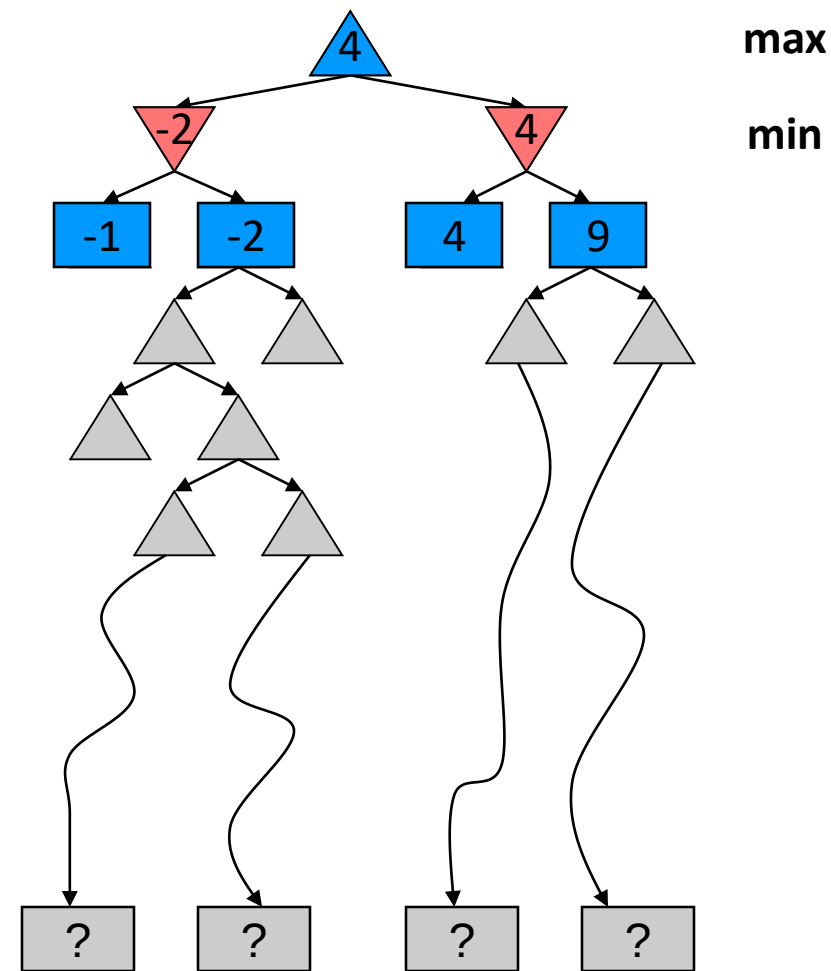


Giới hạn tài nguyên

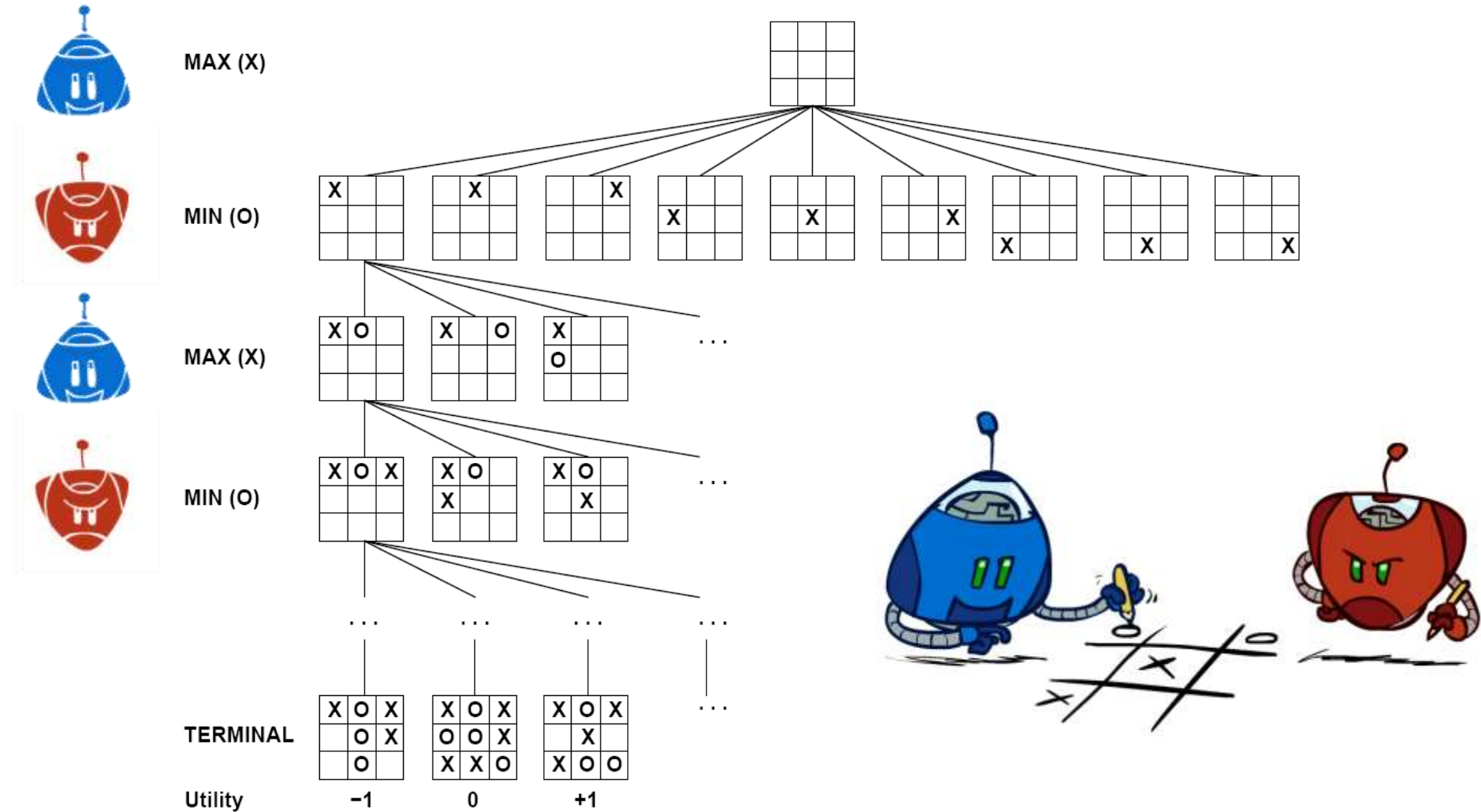


Giới hạn tài nguyên

- Vấn đề: Thực tế: thường không thể tìm kiếm đến lá!
- Giải pháp: Giới hạn độ sâu tìm kiếm
 - Chỉ tìm kiếm đến một độ sâu giới hạn
 - Thay thế hàm tính lợi ích (trạng thái kết thúc) bằng hàm ước lượng lợi ích (cho trạng thái không kết thúc)
- Ví dụ: Chess
 - Giả sử giới hạn thời gian 100 giây, mỗi giây mở rộng 10 nghìn trạng thái
 - Có thể tính 1 triệu trạng thái cho mỗi lượt đi
 - α - β có thể tính đến độ sâu 8
- Tính tối ưu không được đảm bảo
- Độ sâu tăng = tác tử càng giỏi
- Sử dụng tìm kiếm độ sâu tăng dần (để thoát khi cần)

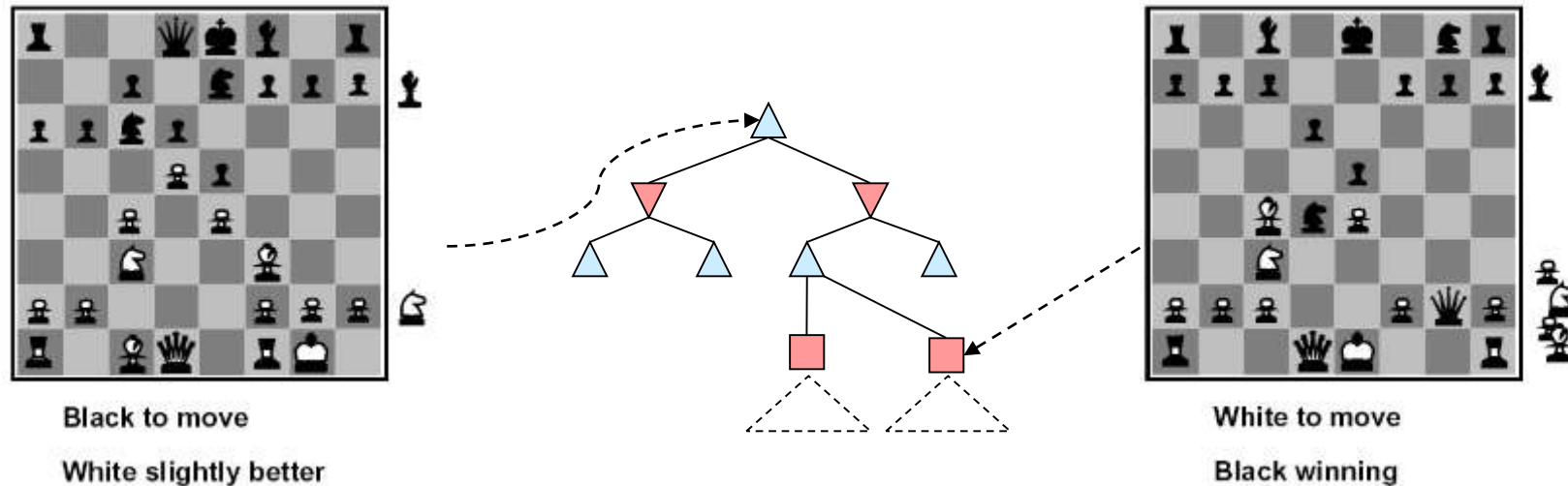


Hàm giá trị cho Tic-Tac-Toe



Hàm ước lượng giá trị

- Hàm ước lượng giá trị của trạng thái (không kết thúc)



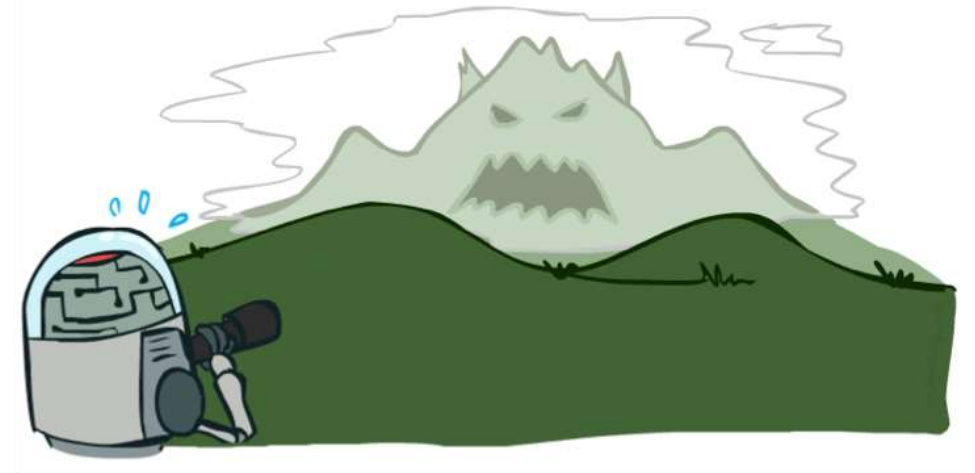
- Ước lượng lý tưởng = giá trị minimax
- Thực tế = tổng có trọng số của các đặc trưng

$$Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

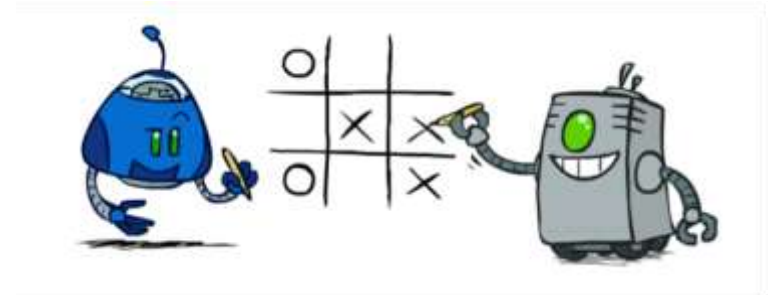
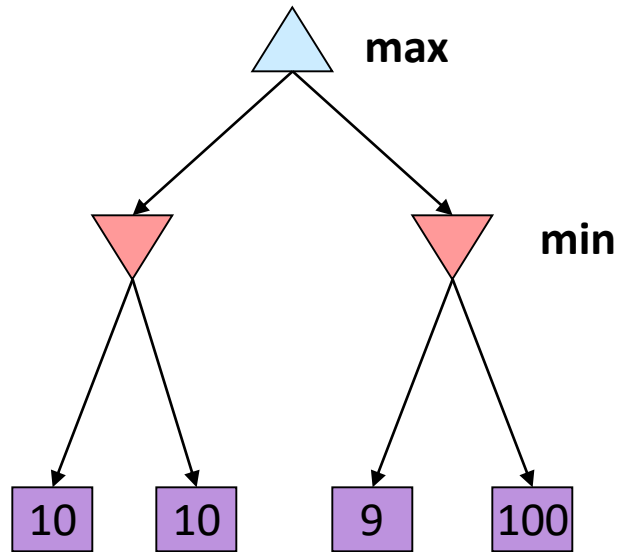
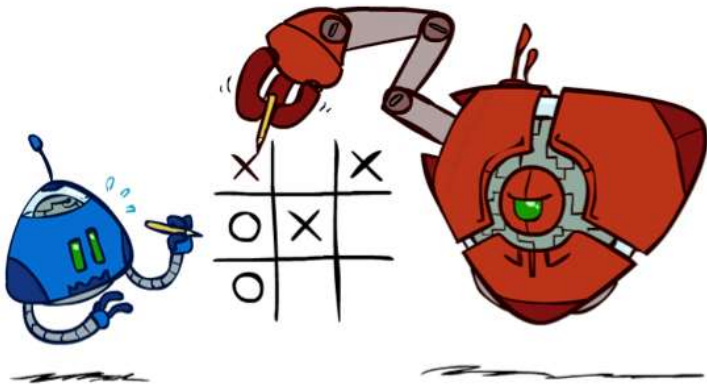
- Ví dụ: $f_1(s) = (\text{Số hâu trắng} - \text{số hâu đen}), \text{ v.v...}$

Độ sâu tìm kiếm và hàm ước lượng

- Hàm ước lượng không bao giờ hoàn hảo
 - Định hướng mở rộng các nút “hứa hẹn” -> có sự tương quan với hàm kinh nghiệm trong A^*
- Độ sâu lớn hơn thì ước lượng dễ hơn?
- Đánh đổi giữa
 - Độ phức tạp đặc trưng trò chơi
 - Ước lượng sát với đặc trưng
 - Độ phức tạp tính toán của hàm ước lượng



Minimax: giả thiết về đối thủ



Tối ưu khi chơi với đối thủ tối ưu. Nếu đối thủ không chơi tối ưu?

Bài tập

- Cài đặt Tic-Tac-Toe
- Cài đặt trò chơi cờ ca rô (tự đặt giới hạn kích thước)
- Tìm hiểu giải thuật các trò chơi như cờ vua, cờ tướng, cờ vây,...

Đọc thêm: Monte Carlo Search Tree

- Tìm kiếm dựa trên xác suất và mô phỏng
- Phù hợp với các bài toán mà chưa có tri thức/kinh nghiệm tường minh
- Có thể dùng online trong pha tìm kiếm hoặc offline (kết hợp với một phương pháp học máy)